Inside the Model Codebase Report

Repository overview and implementation status | 6 October 2026

Purpose and current state

Inside the Model is a free, no-login learning website that explains AI through interactive visualizations and small models running in the browser. Its 15-day curriculum progresses from learning fundamentals to frontier topics, with a separate math primer. The codebase has substantial teaching content through reinforcement learning on Day 11; alignment and most later topics remain planned. It is a static educational application with no application backend, database or authentication service. [1, 2]

Architecture and organization

Astro 7 generates static routes and lesson pages. MDX combines prose, equations and embedded widgets; Svelte 5 islands provide browser interaction. Strict TypeScript supports the numerical libraries and component logic. KaTeX renders mathematics during the build. The content schema validates track, order, level, duration, prerequisites and lesson status, and requires review dates for frontier lessons. [1, 2]

Area	Responsibility
src/pages and src/layouts	Landing page, curriculum map, lesson routes, navigation and shared page structure.
src/content/lessons	MDX teaching content organized by track and lesson slug.
src/components/widgets	Svelte interfaces, pure sim.ts mathematics, workers and shared figure controls.
src/lib	Curriculum, SEO, refresher cards, numerical engines and model loading.
public/models and scripts	Model binaries and manifests, offline training, browser checks and budget checks.
tests/unit	Vitest coverage for algorithms, simulations and shared infrastructure.

Curriculum and source inventory

98 lessons across 12 tracks: 72 draft, 26 stub, zero marked done. The source contains 66 widget directories with matching Svelte and simulation files, 73 unit-test files, 11 worker.ts files and six model binaries. Lesson MDX contains 408 hydration directives. These are file and metadata counts, not evidence that every feature has passed review.

Written material covers the math primer, foundations, classic machine learning, neural networks, CNNs, sequences, transformers, language models and five reinforcement-learning lessons. One graph-neural-network lesson is also drafted. The remaining stubs comprise six alignment lessons, five graph/generative lessons, seven applied-AI lessons and eight frontier lessons. Documentation records coordinator acceptance through Day 11, while owner clarity review remains pending for Days 6-11. Draft metadata and coordinator acceptance are separate states. [2, 5]

Implementation Quality and Remaining Work

Numerical foundations, verification evidence and development priorities

Shared mathematics and model pipeline

The neural-network core includes an inspectable scalar autograd graph, multilayer perceptrons, losses and optimizers. The CNN core supplies convolution and pooling operations with backward passes. A shared transformer engine supports attention traces, training gradients, generation and cached decoding; language-model helpers add sampling and operation-cost accounting. Reinforcement-learning modules implement gridworlds, bandits, tabular methods, DQN and PPO. Widgets reuse these engines so the same mechanism can be inspected from several lesson views. [3]

Offline training produces compact binary weights and JSON manifests under public/models. The loader supports float16 and int8 representations, validates model structure, checks SHA-256 when supplied and memoizes requests. Some lessons train small models in browser workers. This keeps the learning experience self-contained while separating expensive computation from interface rendering. [3]

Interaction and delivery

Shared figure cards and timelines provide playback, scrubbing, speed selection and readouts. Pure simulation functions and seeded randomness support reproducible states. Animation helpers pause when figures or tabs are hidden and expose reduced-motion support. RL widgets use lazy mode modules and shared worker helpers. Light/dark themes use semantic CSS tokens; fonts are local. Lesson pages include prerequisite links, refresher summaries, optional equations and previous/next navigation. [4]

The SEO foundation includes canonical URLs, social metadata, structured learning-resource data, a sitemap and robots output. The configured site URL is inside-model.vercel.app. This report describes the current local checkout, which includes uncommitted work; it does not establish that the deployed site matches these files. [1, 4]

Verification evidence and practical limits

Vitest tests cover numerical invariants, deterministic behavior and gradients. Astro diagnostics check types and content. Repository scripts inspect curriculum consistency, compressed widget closures and browser behavior, including mobile layouts, themes, console errors, overflow and per-step stalls above 100 ms. The recorded Day 11 checkpoint reports **811 passing tests in 73 files**, zero Astro diagnostics, a 104-page build and clean checks on its five lessons. GridWorldLab reached 39.7 KB gzip, close to the widget budget. [5]

A later saved audit records 72 drafted lessons, 408 figures and 31 initial findings involving repeated figures, unclear explanations, interaction behavior and performance. It records subsequent fixes and 824 passing tests at an intermediate checkpoint, while retaining remaining items. These are historical records; no application tests, build, browser audit or performance benchmark was rerun for this report. Technical success alone does not establish teaching clarity. [5, 6]

Remaining work and priorities

Complete Day 12 alignment and its shared core before Days 13-15, following the existing plans. Resolve and recheck outstanding audit items, including repeated figures and cold-start performance. The glossary and aggregate refresher pages are placeholders, although individual lesson refresher cards are implemented. Search and learner-progress tracking are absent, presentation mode is partial, and no production CI workflow or implemented E2E suite was found. Keep source claims, visible widget states and explanatory prose consistent as the remaining curriculum is written. [4-6]

Source index [1] package.json, astro.config.mjs, tsconfig.json. [2] src/content.config.ts, src/lib/curriculum.ts and lesson MDX inventory. [3] src/lib/{nn,cnn,transformer,lm,rl}, scripts/train-models.ts and public/models. [4] layouts, pages, widgets/_shared, widgets/_rl, styles and src/lib/seo.ts. [5] BUILDING.md, plans/README.md, tests/unit and verification scripts. [6] plans/audit-2026-10-05.md.